

Fundamentals of Building AI Agents

Author: Dr. Wei Liu

Edition: 2026 V1

1. Preface

This comprehensive guide is designed for developers, data scientists, and AI practitioners seeking to master the engineering of autonomous systems. Unlike traditional software, AI agents introduce non-deterministic behaviors that require robust architectures, precise state management, and rigorous evaluation.

This edition synthesizes practical insights from project-specific materials and authoritative web sources (including LangChain, Anthropic, and Microsoft), offering a neutral, technical perspective on the evolving landscape of agentic AI.

2. Foundations: What Is An AI Agent?

At its core, an **AI Agent** is a system that uses a Large Language Model (LLM) as a reasoning engine to determine control flow and interact with external environments. While a standard LLM answers questions based on training data, an agent actively takes actions—executing code, querying databases, or calling APIs—to achieve a goal.

The distinction between a **Workflow** and an **Agent** is pivotal. A workflow follows a predefined path (DAG), whereas an agent dynamically decides its next step based on observation.

Feature	Workflow (Chain/Pipeline)	AI Agent
Control Flow	Hardcoded / Static	Dynamic (Model-driven)
Predictability	High (Deterministic)	Variable (Non-deterministic)
Best For	Well-defined, repetitive	Ambiguous, multi-step

	tasks	problem solving
Error Recovery	Requires manual exception handling	Self-correcting via loops

Decision Checklist: When to Use an Agent?

Use this checklist to determine if your use case requires an agentic architecture:

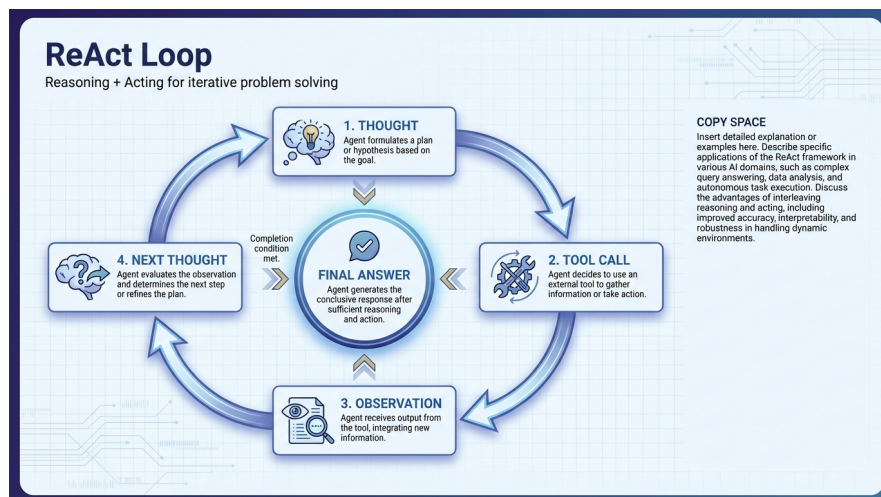
- Does the task require dynamic decision-making based on intermediate results?
- Is the sequence of steps unknown beforehand?
- Does the system need to interact with multiple external tools (APIs, DBs) flexibly?
- Is the outcome open-ended (e.g., "Research this company") rather than specific (e.g., "Extract the date")?

3. Agent Architectures and Core Components



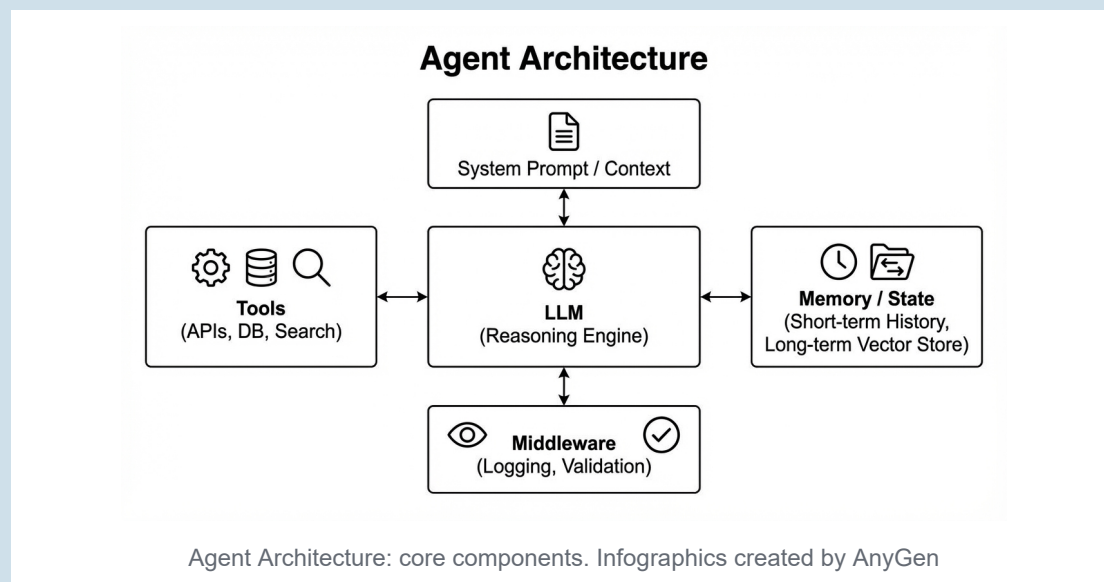
The LangChain Ecosystem supports both explicit Chains and dynamic Agents.

Modern agents typically follow the **ReAct** (Reasoning + Acting) pattern. In this loop, the model: 1) Thoughts (Reasons about the current state), 2) Actions (Decides to call a tool), and 3) Observations (Receives tool output).



ReAct Loop: Reasoning + Acting cycle. Infographics created by AnyGen

Core Components



- **The Brain (LLM)**: The reasoning engine responsible for planning and tool selection. It must be capable of [Structured Output](#) to reliably invoke functions.
- **Tools**: Interfaces to the outside world (Calculators, Search, APIs). In LangChain, these are often defined using the `@tool` decorator.
- **Memory (State)**: Short-term memory (conversation history) and long-term memory (vector stores) allow the agent to maintain context across multiple turns.
- **Middleware**: The control layer that handles logging, error catching, and safeguards before and after the LLM execution.

4. Workflows vs Agents: Patterns and Trade-offs

Agents vs Workflows		
	 Agents	 Workflows
Control Flow	 Dynamic Adaptive paths based on real-time data, context, and goals. Decision-making is decentralized and evolves.	 Static Pre-defined, fixed sequence of steps. Paths are determined beforehand and followed rigidly.
Predictability	 Variable Outcome fluctuates based on changing conditions and internal logic. Less certain.	 Deterministic Consistent outcome given the same inputs. Highly predictable and repeatable.
Best For	 Ambiguous tasks Complex decision-making, problem-solving in uncertain environments, personalization, exploration.	 Well-defined tasks Repetitive processes, data migration, standard operating procedures, approval chains, rule-based operations.
Error Recovery	 Self-correcting loops Autonomous detection and resolution; adjusts strategy to overcome obstacles without human intervention.	 Manual exceptions Stops upon error, requires human intervention or predefined exception handling paths to resume.

Agents vs Workflows: key differences. Infographics created by AnyGen

While agents are powerful, they are expensive and slower. [Workflows](#) offer a middle ground. Below are common patterns.

Common Patterns

Prompt Chaining

Sequential steps where the output of one LLM call becomes the input for the next. Ideal for clear, linear tasks like "Translate then Summarize".

Routing

The LLM classifies an input (e.g., "Refund Request" vs "Tech Support") and directs it to a specialized downstream workflow.

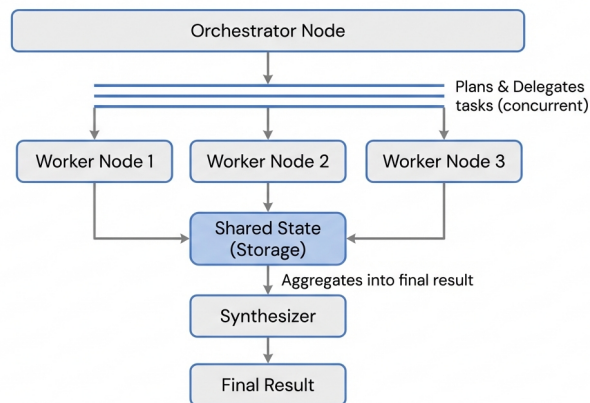
Orchestrator-Worker

A central "Orchestrator" breaks a complex task into subtasks and delegates them to specific "Worker" modules, synthesizing the results at the end.

Evaluator-Optimizer

A loop where one LLM generates a solution and another acts as a critic/grader. If the grade is low, the generator retries with feedback.

Orchestrator-Worker Pattern



Orchestrator-Worker pattern. Infographics created by AnyGen

5. Hands-on: Building a LangChain Agent

Building a basic agent involves defining tools, selecting a model, and binding them together. Below is a simplified example using LangChain.

Python

```
from langchain.tools import tool
from langchain_anthropic import ChatAnthropic
from langgraph.prebuilt import create_react_agent

# 1. Define Tools
@tool
def multiply(a: int, b: int) -> int:
    """Multiply two integers."""
    return a * b

@tool
def search_web(query: str) -> str:
    """Simulated search tool."""
    return f"Results for {query}..."

tools = [multiply, search_web]

# 2. Initialize Model
llm = ChatAnthropic(model="claude-3-sonnet-20240229")

# 3. Create Agent (ReAct Loop)
```

```
agent_executor = create_react_agent(llm, tools)

# 4. Invoke
response = agent_executor.invoke({
    "messages": [("user", "What is 123 * 456?")]
})
print(response["messages"][-1].content)
```

6. Structured Outputs and Manual Tool-Calling

Sometimes you need precise control over the execution flow. Instead of letting the agent loop automatically, you can perform **Manual Tool Calling**. This is critical for safety and validation.

By using `bind_tools` and inspecting the `tool_calls` attribute of the response, developers can verify inputs before execution.

Step-by-Step Manual Execution

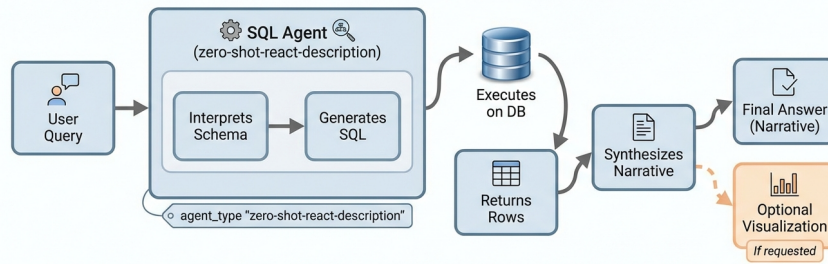
1. **Bind Tools:** `llm_with_tools = llm.bind_tools([tool1, tool2])`
2. **Invoke LLM:** Get the raw message. Check `msg.tool_calls`.
3. **Validate:** Ensure parameters (e.g., SQL queries) are safe.
4. **Execute:** Run the tool function manually.
5. **Feedback:** Pass the tool output back to the LLM for the final response.

7. Data-Connected Agents: SQL and Visualizations

Agents can bridge natural language and structured data. AI-powered SQL Agents interpret schemas and generate queries dynamically.

SQL Agent Implementation

SQL Agent Pipeline



SQL Agent Pipeline (zero-shot-react-description). Infographics created by AnyGen

Python

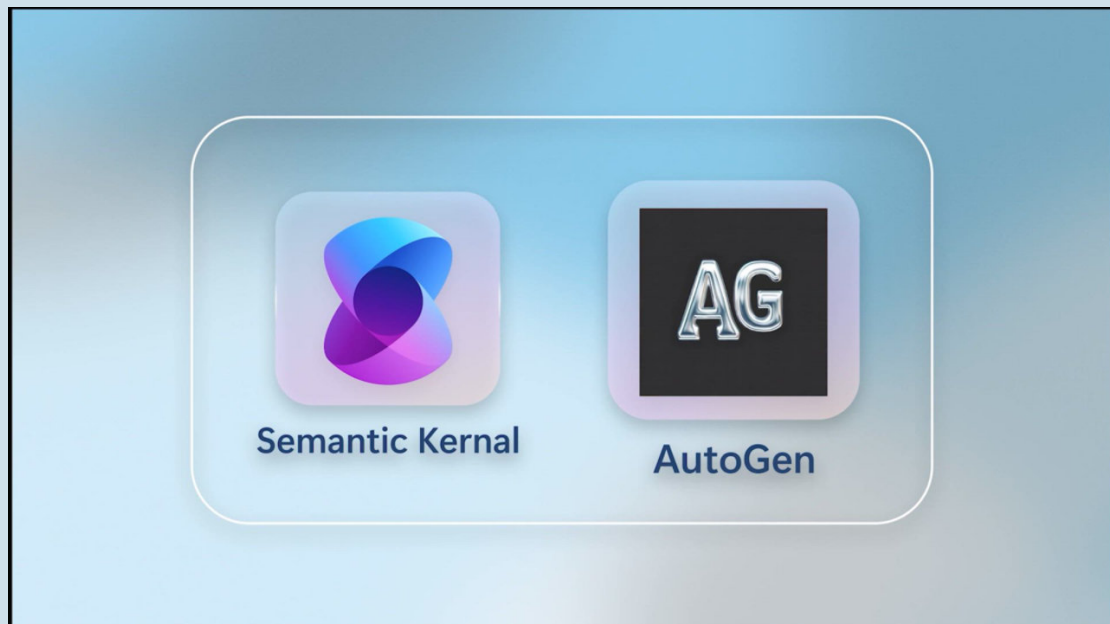
```
from langchain_community.agent_toolkits import create_sql_agent
from langchain_community.utilities import SQLDatabase

db = SQLDatabase.from_uri("sqlite:///Chinook.db")
agent = create_sql_agent(llm, db=db, agent_type="zero-shot-react-
description", verbose=True)

# The agent inspects schema, generates SQL, executes, and
synthesizes answer
agent.invoke("How many albums are in the database?")
```

For visualization, the **Pandas Agent** can generate Python code to plot data. Note that this requires a sandboxed environment for safety, as the agent executes generated Python code.

8. Frameworks Landscape (2026)



The Microsoft Agent Framework offers structured workflows for enterprise agents.

Framework	Key Philosophy	Best For
LangChain / LangGraph	Graph-based state management. Explicit control over cycles.	Complex, stateful production apps requiring fine-grained control.
OpenAI Responses API	Migrated from "Assistants". Unified API for tools and retrieval.	Quick deployment with hosted infrastructure.
Microsoft Agent Framework	Focus on multi-agent collaboration and enterprise integration.	Enterprise environments using the Azure ecosystem.

9. Safety, Guardrails, and Human-in-the-Loop

Deploying agents requires rigorous safety protocols. The non-deterministic nature means agents can loop indefinitely or call tools incorrectly.

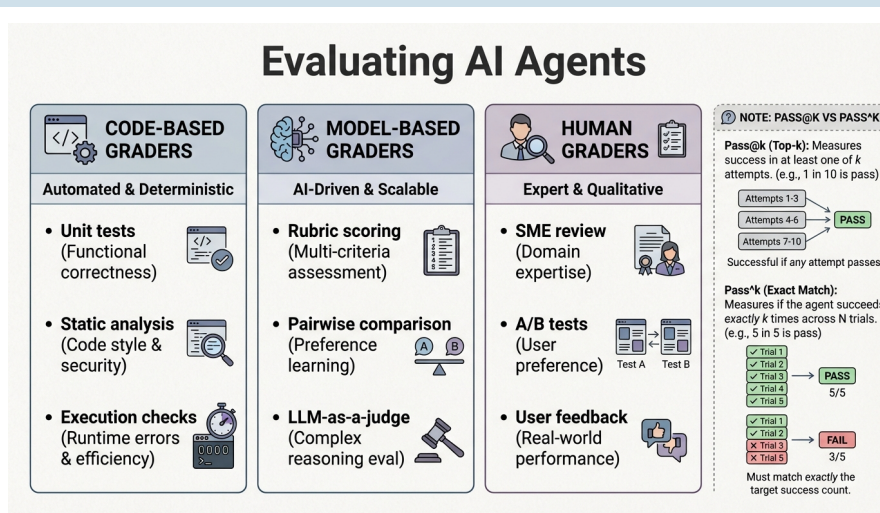
Safety Layers

- **Middleware Safeguards:** Intercept input/output to check for PII, toxicity, or hallucinated URLs.

- **Human-in-the-Loop (HITL):** Use [interrupts](#) before sensitive tool calls (e.g., executing a refund). The agent pauses and waits for human approval.
- **Checkpoints:** Save the state of the agent at every step. This allows "Time Travel"—rewinding an agent to a previous state to fix an error.

10. Evaluating AI Agents

Evaluation is the bedrock of reliable agents. According to [Anthropic's research](#), simple unit tests are insufficient. You need a mix of graders.



Evaluation Graders: code-based, model-based, human. Infographics created by AnyGen

Evaluation Metrics

Pass@ k : The probability that at least one solution is correct in k attempts. Useful for creative tasks. **Pass^ k :** The probability that *all* k attempts are correct. Essential for reliability-critical tasks.

Sample Evaluation Task (YAML)

YAML

```
task:
  id: "fix-auth-bypass"
  graders:
    - type: deterministic_tests
      required: [test_security.py]
    - type: llm_rubric
      rubric: prompts/code_quality.md
  tracked_metrics:
```

```
- type: transcript
  metrics: [n_turns, n_toolcalls]
```

11. Best Practices and Production Checklist

Moving from prototype to production requires discipline. Use this 20-point checklist to ensure readiness.

Production Checklist

- **Planning:** Are the agent's goals and scope clearly defined?
- **Architecture:** Is the choice between Workflow vs Agent justified?
- **Tooling:** Are all tools idempotent and safe to retry?
- **Memory:** Is there a mechanism to prune context to avoid context-window overflows?
- **Evaluation:** Do you have a regression suite (pass@1 > 90%)?
- **Safety:** Are sensitive tools gated by HITL?
- **Observability:** Are you logging full traces (LangSmith/Phoenix)?

12. Glossary

- **Agent:** A system using an LLM to decide control flow and tool usage.
- **ReAct:** A widely used approach that interleaves reasoning traces with actions (tool calls) to iteratively solve tasks.
- **Middleware:** Software that wraps the LLM to provide logging, state, and safety.
- **MCP (Model Context Protocol):** A standard for connecting AI assistants to systems of record.
- **Safeguards (Guardrails):** policies, filters, and validation layers used to constrain model behavior and tool usage.

13. Appendix: Sources and Citations

The following sources were used in the compilation of this guide:

- [Workflows and Agents - LangChain Docs](#)
- [Demystifying Evals for AI Agents - Anthropic](#)
- [LangChain Agents Documentation](#)